

Quarto Workshop

From data to formatted publication

Patric Eichelberger

patric.eichelberger@bfh.ch

Bern University of Applied Sciences

22.07.2026

Quarto lets us...

- Analyze data and visualize data through R and Python (and more): **You know - the things we do anyway**
- Create dynamic content for scientific reports, manuscripts or presentations: **Create output when input changes**
- Using one source and produce output in various formats like HTML, PDF and MS Word: **Reduce copy-paste**
- Create reports, manuscripts, presentations, websites, books or knowledge repos through [Pandoc](#) and writing in [Markdown](#)
- Do all Open-Source and adopt good Open-Science practice

Provide seamless and reproducible workflows from data analysis to dissemination of knowledge

Aims of this workshop

- After this workshop you can understand and describe the basic mechanisms of Quarto and know where it can support your research activities.
- Apply Quarto to concrete use-cases to get inspiration about its capabilities.
- You will have a running toolchain on your computer to work further on your own projects.

What we don't aim

- Introduce coding basics, R programming or Markdown editing.
- Workshops to R and ggplot were given in the past and resources are available [in this Gitlab repository](#)
- Quarto is huge. We won't cover everything.

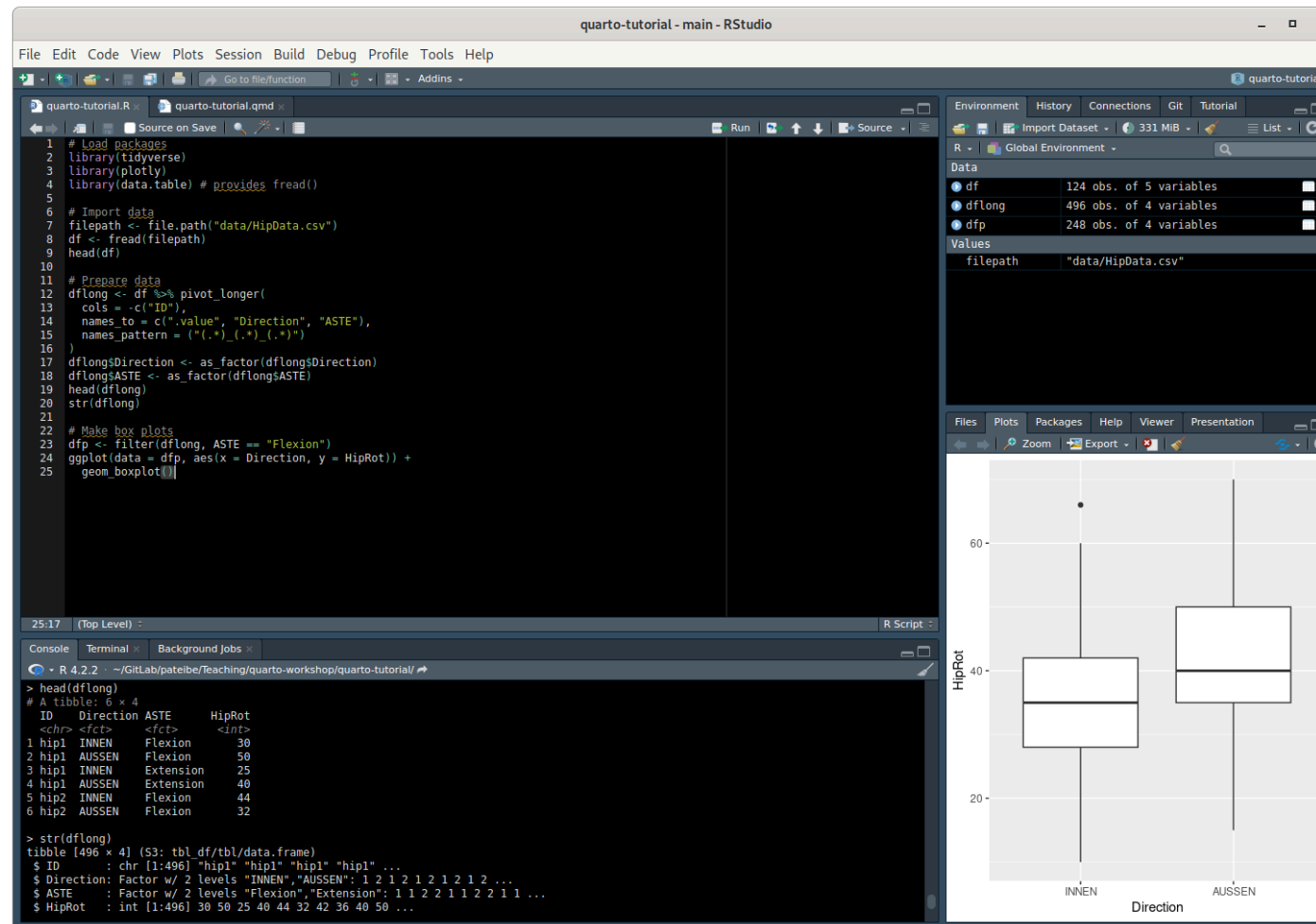
Helpful resources and tips

- [Quarto website](#). Lot of material > good to take time to familiarize yourself. The [Guide](#) section is a good starting point.
- Adopt using [Visual Studio Code](#), a powerful text editor.
- Check out the getting started tutorials for [VS Code](#) or [RStudio](#) on the Quarto website.
- Familiarize with basic [Markdown authoring](#).
- [RStudio approach](#) is easier to start for R users.
- [VS Code workflow](#) is more versatile but harder to adopt.

Workshop tools

- R
- RStudio
- Visual Studio Code
- Quarto

R-Script (.R)



Outputs go to the console and to the plot pane.

Quarto document (.qmd)

The screenshot shows the RStudio interface with a Quarto document open. The code in the editor is as follows:

```

1 ---
2 title: "quarto-tutorial"
3 author: "Patric Eichelberger"
4 format: html
5 ---
6
7 # Load packages
8
9 ##(r)
10 library(tidyverse)
11 library(plotly)
12 library(data.table) # provides fread()
13
14
15 # Import data
16
17 ##(r)
18 filepath <- file.path("data/HipData.csv")
19 df <- fread(filepath)
20 head(df)
21

```

The console output shows the execution of the code, including the loading of packages, the import of data, and the creation of a boxplot. The environment pane shows the loaded data frames:

- df: 124 obs. of 5 variables
- dflong: 496 obs. of 4 variables
- dfp: 248 obs. of 4 variables

The boxplot shows the distribution of HipRot for different directions and extensions. The x-axis is labeled 'Direction' and the y-axis is labeled 'HipRot'. The legend indicates that the boxplots are colored by 'Extension' (INNER, OUTER).

- Outputs are inline within the `.qmd` file when run (Button **Run**) in RStudio.
- Text, code and output go to `.html` (or whatever format we set in the YAML-header) when rendered through Quarto (Button **Render**).

How it works?

qmd file with R code in RStudio



qmd file with Python code from VS Code (or command line)



YAML header

Where we set rendering options

```
1 ---
2 title: "quarto-tutorial"
3 author: "Patric Eichelberger"
4 format: html
5 ---
```

- Common and format-specific rendering options.
- Multiple formats can be specified to produce different output from the same source.

More complex

```
1 ---
2 toc: true
3 bibliography: references.bib
4 lang: de
5 date: today
6 csl: ieee.csl
7 format:
8   pdf:
9     documentclass: scrreprt
10    papersize: a4
11    number-sections: true
12    margin-left: 30mm
13    margin-right: 30mm
14    mainfont: Unit Rounded Pro
15    mainfontoptions:
16      - UprightFont = *-Light
17   html:
18     toc: true
19     embed-resources: true
20 ---
```

Computations

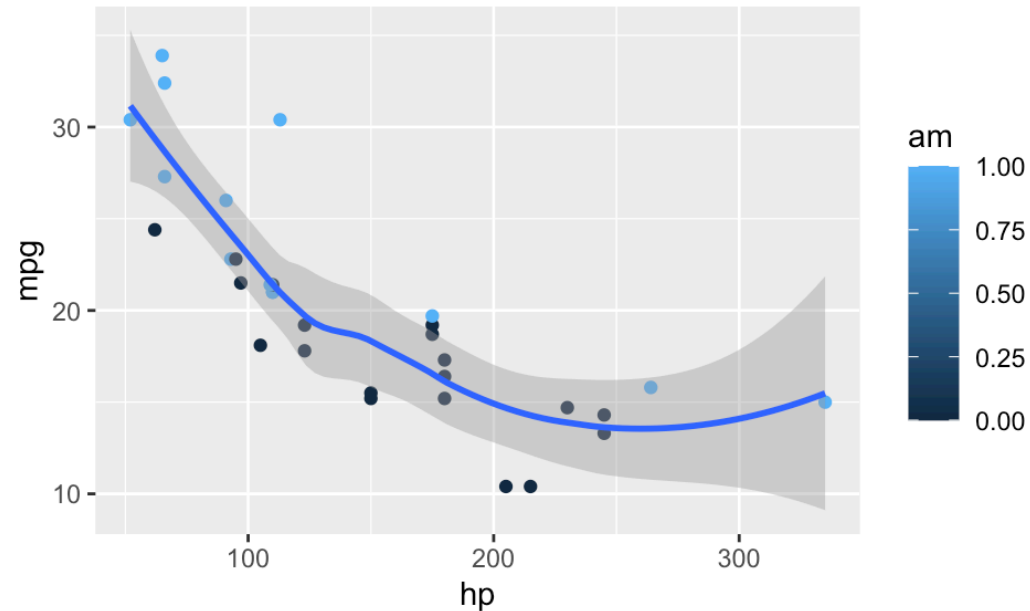
- Computations are carried out within code chunks.
- Execution options can be set with `# |`.

```
1  ```{r}
2  #| label: fig-hiprotation # the chunk can be referred to by its label
3  #| fig-cap: "Hip internal and external rotation." # caption
4  #| eval: true # evaluate chunk when set to true
5  #| include: true # include output when set to true
6  #| warning: false # include warnings in the output
7  #| fig-width: 5 # set figure width to 5 inches
8
9  print("Hello World!")
10 ```
```

```
[1] "Hello World!"
```

Example dynamic content

```
1 library(ggplot2)
2 ggplot(mtcars, aes(hp, mpg, color = am)) +
3   geom_point() +
4   geom_smooth(formula = y ~ x, method = "loess")
```



The plot is directly created during rendering and not imported from an external file.